



## Build a Smart Assistant That Watches Shipping Costs

You'll get this brief 24 hours before Round 2. Come ready to demo and walk through your solution -- reasoning and trade-offs matter as much as working code.

### THE PROBLEM, IN PLAIN TERMS

Companies pay transporters to move goods. Cost usually depends on how far the truck travels and how much it is carrying. Over time, that cost can quietly creep up on certain routes -- sometimes for a real reason (fuel got pricier, a toll changed, monsoon disruption), sometimes for no good reason at all. Your job: build a smart assistant that spots when a route's cost is rising in a way that doesn't look justified, and explains why in plain English.

### WHAT YOUR ASSISTANT MUST DO (CORE)

- Group shipment records by route (same start and end point) and by the **route\_type** field already provided (Short / Medium / Long) -- you don't need to derive length buckets yourself.
- For each group, track cost per tonne per km, week by week.  $\text{Cost per tonne per km} = \text{total freight cost} / (\text{quantity carried in tonnes} \times \text{distance travelled in km})$ .
- Weeks run Monday to Sunday; **week\_of** is that week's Monday date -- group **shipment\_date** into these weeks before computing anything.
- Compare this week's cost against what this route used to cost in the past, and what similar-length routes currently cost -- see "Comparison baselines, precisely" below for the exact definitions we'll grade against.
- Flag any route where the cost is rising and looks out of the ordinary compared to its own history or to similar routes.

#### *Comparison baselines, precisely*

*These two comparisons are numeric fields we grade, so use these definitions exactly -- don't improvise your own window:*

- **vs. own history:** the route's trailing 8-week rolling average cost per tonne-km, using only weeks strictly before the current week (no look-ahead). If fewer than 8 prior weeks exist, use all prior weeks available and say so in your README -- don't pad or extrapolate.
- **vs. similar routes:** the average cost per tonne-km, in that same week, across all other routes that share the same route\_type (Short / Medium / Long). Exclude the route itself from its own peer average.

### MAKE IT SMARTER; THIS IS WHERE AI COMES IN (BONUS)

- Before flagging a route, have your assistant check a small set of notes which include fuel price changes, toll updates, weather disruptions, and festival surcharges, to see if there's a real reason behind the cost rise. This is a great place to use a vector database and retrieval (RAG).
- Instead of just showing a flag, write a short, plain-English explanation for each one, backed by what it actually found in those notes -- it should not be made up.
- **Stretch goal:** let someone ask your assistant questions about its findings in plain English (e.g. "why did this route get pricier in March?") and have it answer using its own results.

### GUARDRAILS WE WILL TEST

A working demo isn't the bar. We will probe whether your assistant can be trusted, not just whether it runs.

#### **Hallucination**

- Every "justified" verdict must cite the exact note, and that note must apply to that route and that time. Wrong route, wrong window, or a note that says costs weren't affected does not count.
- No supporting note -> unexplained. Do not guess; a confident wrong reason scores worse than "couldn't find one."
- We will spot-check justified rows against the given notes. Failed checks are marked fabricated even if the flag was right.

#### **Knowing your system is right**

Show a check -- a lightweight eval harness or a small labeled set you score against -- and why you trusted the result enough to submit.



### Reproducibility

- Three untouched runs on the same input: identical flags and numbers. Explanation wording may vary; cited note, verdict, and numbers must not.
- If anything is random (e.g. an LLM call above temperature 0), say what you set, why, and show it does not move the verdict.

### Cost of running it

- Prefer open-source models and free-tier options. For one full run over the entire shipment file, report total input tokens, total output tokens, number of LLM calls, and the rough cost at your model's published rate.

## WHAT YOU'LL BE GIVEN

- **shipment\_records.csv** -- sample shipment records (not real company data): origin, destination, route type, material, quantity, distance, cost, and date.
- **context\_notes.csv** -- short notes on real-world reasons costs might change. This is what your AI should search through.
- **sample\_output\_format.csv** -- the exact column structure your output must match. We cannot score submissions that don't follow this format.
- Column names, order, and types in **sample\_output\_format.csv** are the contract, including **matched\_note\_id**.
- **reason** may be in your own words: state the verdict; if justified, name the note and reflect its content and dates. Put the note id in **matched\_note\_id** (blank if unexplained) -- not only in the prose.

## WHAT YOU NEED TO HAND IN

- Your working code (any language or tools you're comfortable with).
- Your output as a CSV matching **sample\_output\_format.csv** exactly.
- A short README explaining how it works, how you built it, and how you convinced yourself it was right before submitting.
- Your reproducibility check across 3 runs (a diff, or a note confirming they matched).
- Your token/cost log for one full run.
- A short (10 min) walkthrough at the start of Round 2.

## HOW WE'LL EVALUATE YOU

| Criterion                    | What we're checking                                                                                          |
|------------------------------|--------------------------------------------------------------------------------------------------------------|
| Does the core logic work?    | Grouping, weekly cost tracking, comparisons, and flags are all correct.                                      |
| Did you use AI meaningfully? | Retrieval pulls up genuinely relevant notes, and explanations are grounded in them and are not hallucinated. |
| Can we trust it?             | Verdicts hold up on a closer read of the notes; nothing invented to fill a gap.                              |
| Does it hold up on repeat?   | Same input, same output every time, no exceptions.                                                           |
| Did you run it responsibly?  | Sensible LLM call volume and an honest token/cost log; not unlimited spend for a fixed answer.               |
| Is the code clean?           | Easy to read and follow, sensibly organised, not just a working mess.                                        |
| Can you explain your work?   | Your README and walkthrough make it easy to follow what you built, why, and how you checked it.              |

## TIMELINE

| BRIEF SHARED BY HR                | SUBMISSION DUE                                             | ROUND 2                                                    |
|-----------------------------------|------------------------------------------------------------|------------------------------------------------------------|
| 24 hours before your Round 2 slot | Before your scheduled Round 2 slot (repo link / zip to HR) | 10-min walkthrough + discussion of approach and trade-offs |

Questions? Reach out to your HR point of contact. Good luck!